Question 1: Convert A + B * C - D to Postfix

Given Expression

A + B * C - D (this is infix form, where the operator is written between the operands)

Operator Precedence

Precedence means which operator gets done first. In this expression there are three operators: +, * and -.

- * has higher precedence than + and -.
- + and - have the same precedence, which is lower than *.
- So B * C is done first, then the + and - are done.

Associativity

Associativity is used when two operators have the same precedence. Here + and - have the same precedence, so we go from left to right. This is called left associativity. That means A + (B * C) is done first, and then D is subtracted from the result.

Operator	Precedence	Associativity
* /	Higher	Left to right
+ -	Lower	Left to right

Method

I used a stack to convert the expression. I read the expression from left to right and followed these rules:

- If it is an operand (A, B, C, D), write it in the output.
- If it is an operator, first remove the operators from the stack that have higher or equal precedence and write them in the output, then push the new operator on the stack.
- At the end, remove all the remaining operators from the stack and write them in the output.

Conversion Table

Step	Symbol	Stack	Output	Action
1	A	(empty)	A	Operand, so write it
2	+	+	A	Stack is empty, so push +
3	B	+	A B	Operand, so write it
4	*	+ *	A B	* is higher than +, so push *
5	C	+ *	A B C	Operand, so write it
6	-	-	A B C * +	* is higher and + is equal, so pop both, then push -
7	D	-	A B C * + D	Operand, so write it
8	End	(empty)	A B C * + D -	Pop the remaining -

Answer

Postfix expression: A B C * + D -

Quick Check

Let A = 2, B = 3, C = 4, D = 5. Infix: 2 + 3 * 4 - 5 = 2 + 12 - 5 = 9. Postfix: 3 4 * gives 12, 2 12 + gives 14, 14 5 - gives 9. Both give 9, so the answer is correct.

Question 2: Binary Search for 6 in [1, 3, 5, 7, 9, 11, 13]

Idea of Binary Search

Binary search can be used only when the array is sorted. Here the array is sorted in ascending order. We find the middle element and compare it with the key. If the key is smaller, we search the left half. If the key is bigger, we search the right half. We repeat this until we find the key or there is nothing left to search.

Array

Index	**0**	**1**	**2**	**3**	**4**	**5**	**6**
Value	1	3	5	7	9	11	13

Key = 6. At the start low = 0 and high = 6. The middle is found using mid = (low + high) / 2.

Simulation

Step	**low**	**high**	**mid**	**arr[mid]**	**Decision**
1	0	6	3	7	6 is smaller than 7, so go left: high = 2
2	0	2	1	3	6 is bigger than 3, so go right: low = 2
3	2	2	2	5	6 is bigger than 5, so go right: low = 3
4	3	2	-	-	low is greater than high, so stop

What Happens

In step 1 the middle element is 7. Since 6 is smaller, we leave 9, 11 and 13. In step 2 the middle element is 3. Since 6 is bigger, we leave 1 and 3. In step 3 only 5 is left, and 6 is bigger than 5, so low becomes 3. Now low (3) is more than high (2), so no elements are left. The search stops and the element is not found (returns -1).

Why

6 is not in the array. If it were there, it would have to be between 5 and 7, but there is no element between them. So the search ends without finding it, after 3 comparisons.